

ENGE707 Week 1

Detailed Theory Notes

These notes explain Lecture 1 in everyday language, aimed squarely at someone who has never studied data or AI before. They focus on what each idea means and why it matters, not on code — the code lives in the slides and labs. Every technical term is introduced with a plain-English version first, and most sections include a real-life example from healthcare, banking, retail, or manufacturing, since the same patterns repeat across all of them.

1. What this lecture promises, and how to actually learn from it

This lecture is the front door to the whole course. Its promise is simple: by the end, you should understand where data comes from, how it gets organised, how AI actually uses it, and why the quality of the data matters more than most beginners expect. AI is not treated as magic here — it's presented as an ordinary, practical workflow that starts with something happening in the real world and ends with a decision, an action, or an alert.

One of the most important ideas in the whole lecture comes right at the start, and it isn't a technical one: the hands-on mindset. In plain terms, this means you don't learn data and AI by reading slides quietly. You learn by typing things in, watching something break, changing a number, running it again, and asking yourself why the answer changed. That loop — try, break, fix, repeat — is genuinely how professionals in this field work day to day, not just how students practice.

***Real-life example:** in a hospital, one nurse's reading of a patient's temperature can become one row in a spreadsheet. In a bank, one tap of a card becomes one transaction record. In a supermarket, one beep of a barcode scanner becomes one line of a sales receipt. Different industries, but the same underlying pattern: something happens in the world, and it gets turned into a piece of stored data.*

2. AI is not just a model — it's a workflow

New students very often jump straight to the question "which model should I use?" — as if picking the right algorithm is the hard part. This lecture pushes back on that instinct hard. A far better first question is: "do we actually have the right data, organised sensibly, with a trustworthy path from the raw input all the way to the final decision?" Get that wrong, and no amount of clever modelling will save you.

The whole journey breaks into four blocks. Real-world data is the raw material — messy, scattered, and not yet useful on its own. Data engineering is the preparation stage that cleans and organises it. AI or machine learning turns the prepared data into a prediction or an output. Application is where that prediction actually does something useful — becomes a dashboard, triggers an alert, or shows up in a report.

Here's the part beginners underestimate: if any early block is weak, everything downstream suffers, no matter how good the model is. A brilliant algorithm fed poor labels, missing measurements, or badly organised data will still produce poor results — it simply has nothing good to work with.

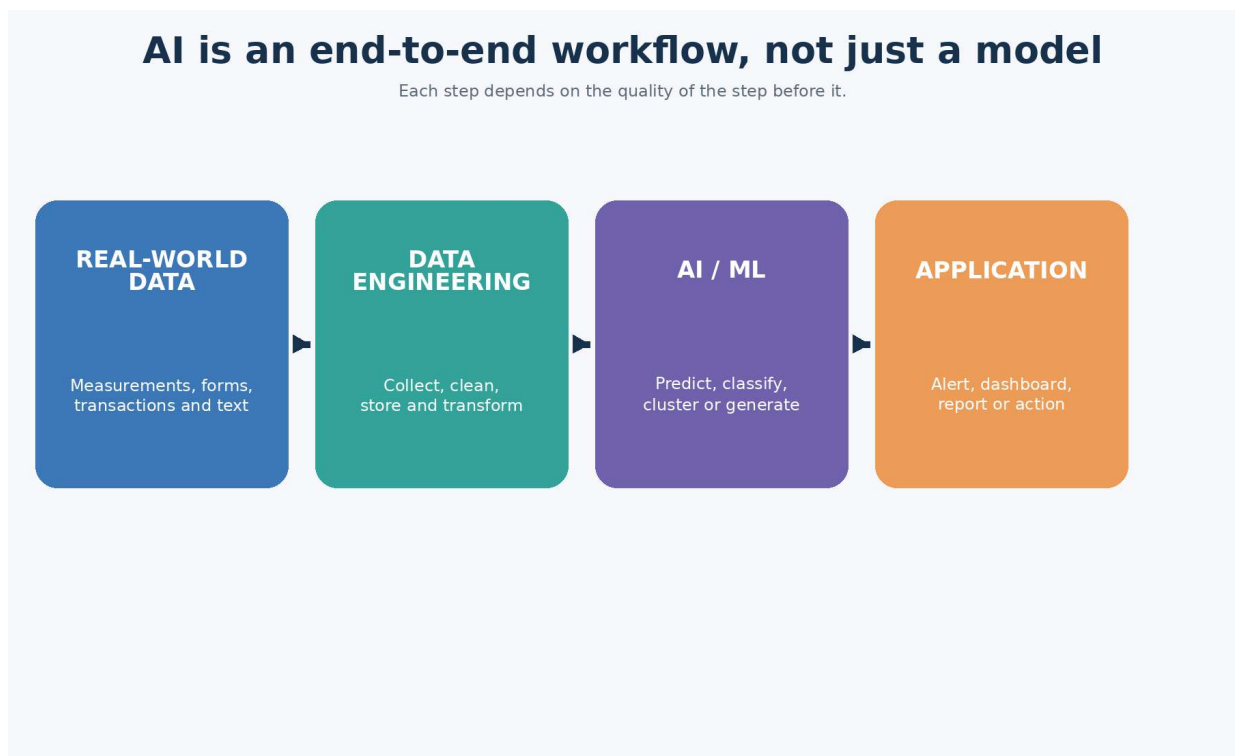


Diagram 1. The four-block workflow: real-world data, data engineering, AI/ML, and application.

3. What real-world data actually looks like

Here's a point that's easy to miss: data doesn't start its life inside a spreadsheet. It starts out in the world, as something that happens. A nurse reads a thermometer. A vibration sensor on a factory machine ticks over. A customer taps their card at a till. Someone posts a comment online. Only afterwards — sometimes seconds later, sometimes much later — does that moment get written down and stored as a record.

In the wound-care example used in this course, a patient's temperature, pH level, and current status become a single line in a CSV file. Swap the domain and the same shape repeats with different measurements.

Real-life example: on a factory floor, a vibration sensor might report a reading every single second. Each one of those readings can eventually become one row in a table used to predict when a machine is about to break down.

4. Data, information and knowledge — three different things

These three words get used interchangeably in everyday speech, but in this course they mean genuinely different things, and the difference matters. Data is a single raw fact on its own — one temperature reading, by itself, means almost nothing. Information is data that's been organised enough to start meaning something — noticing that the average temperature has crept up over several days is information. Knowledge is the next level up: being able to actually use that meaning to make a decision, such as "this patient's rising temperature and rising pH together suggest they're deteriorating."

This matters because a lot of students quietly assume AI starts from "knowledge" — as if the system already understands things the way a person does. In reality, AI almost always starts from raw, messy data and has to climb its way up toward something useful, the same way a human analyst would.

From data to information to knowledge

A raw value becomes useful only after it is organised and interpreted.

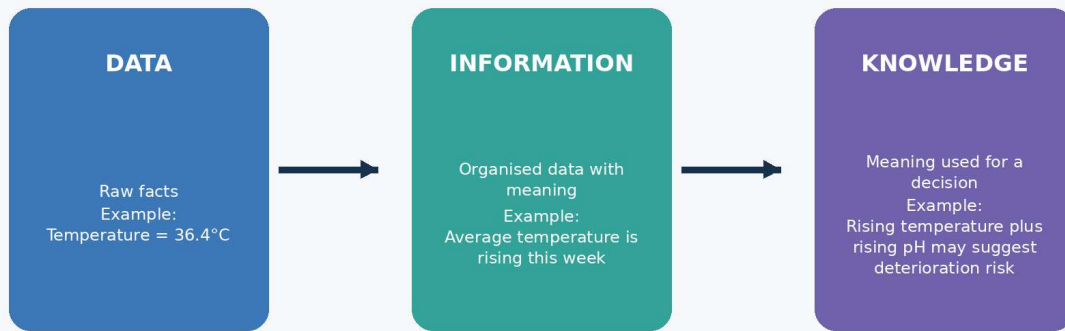


Diagram 2. The difference between raw data, organised information, and useful knowledge.

5. Three types of data: structured, semi-structured, unstructured

Structured data is the tidy, predictable kind — it fits neatly into rows and columns, and you already know in advance what every column means. CSV files and most database tables are structured. Semi-structured data has some organisation to it, but it's more flexible and doesn't force everything into a fixed grid — JSON and XML files are common examples, often used to send data between two computer systems. Unstructured data has no built-in table shape at all — plain text, photos, audio recordings, and video all fall into this bucket.

Worth remembering: this course works mainly with structured, tabular data. Beginners often assume all AI is about recognising photos or chatting like ChatGPT, but in practice, a huge share of real industry AI — fraud detection, demand forecasting, patient risk scoring — starts from nothing more exotic than an ordinary table.

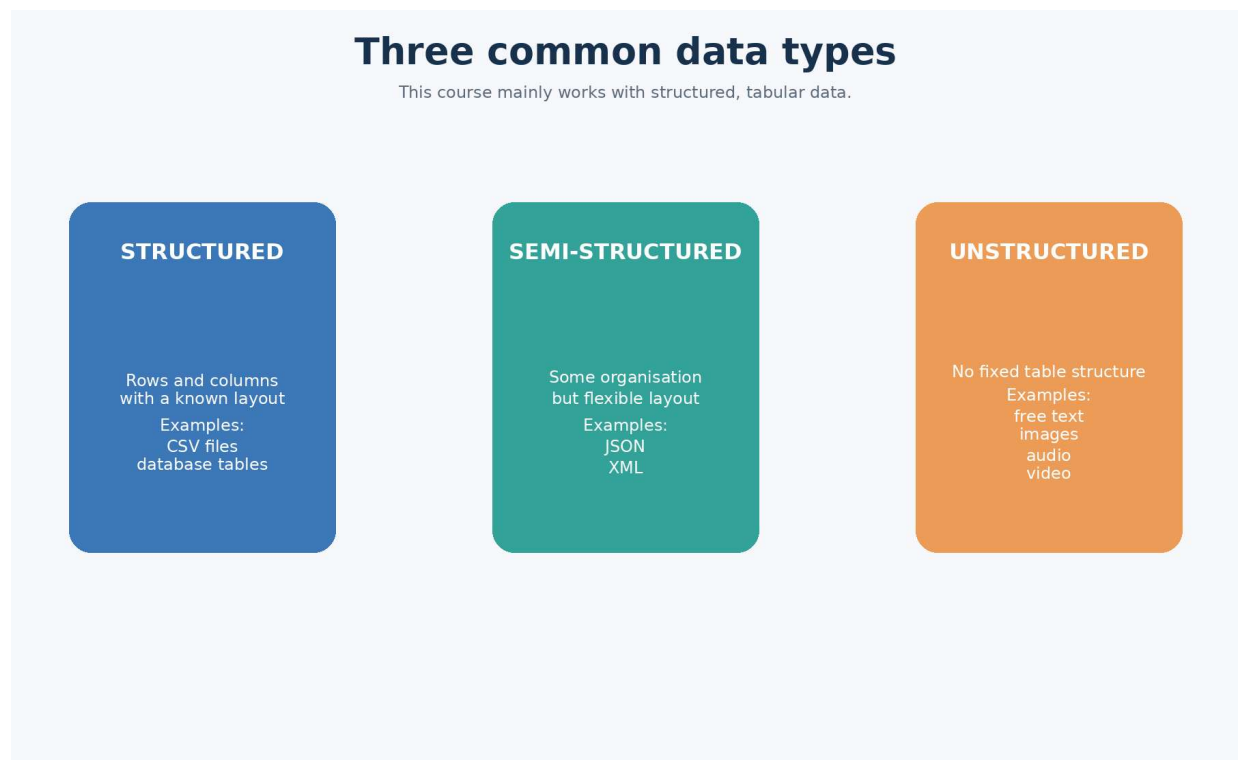


Diagram 3. Three common data types: structured, semi-structured, and unstructured.

6. Data engineering: the essential block before AI ever runs

Data engineering is the work of collecting, cleaning, storing, transforming, and serving data so that it's actually usable. In plain terms, it's the preparation stage — the part where messy, real-world records get turned into something an AI system can actually work with, the same way ingredients need washing and chopping before they go anywhere near a pan.

Even the small classroom steps in this course — opening a CSV file, checking whether any values are missing — are genuine data engineering. They're simple, but they follow exactly the same logic that large organisations use at a much bigger scale.

Real-life example: *a bank might pull transaction records from a dozen separate branch systems overnight, merge them into one central store, strip out duplicates, and standardise the date formats — all before any fraud-detection model is even allowed to look at the data.*

7. Why data is hard to work with: the 4 Vs

The "4 Vs" are a simple, memorable way to describe the practical headaches of working with real data. Volume is just how much data there is. Velocity is how fast it keeps arriving. Variety is how many different shapes and formats it comes in. Veracity is whether you can actually trust it.

For beginners, veracity is usually the one that matters most, and it's the easiest to underestimate. A dataset that's enormous isn't automatically useful if the numbers in it are wrong, biased, or badly measured. Data that arrives instantly isn't valuable either, if it can't be trusted once it gets there.

Real-life example: social media generates data at huge volume and huge speed, but if a large share of posts are spam, duplicated, or deliberately misleading, all that volume and speed buys you very little.

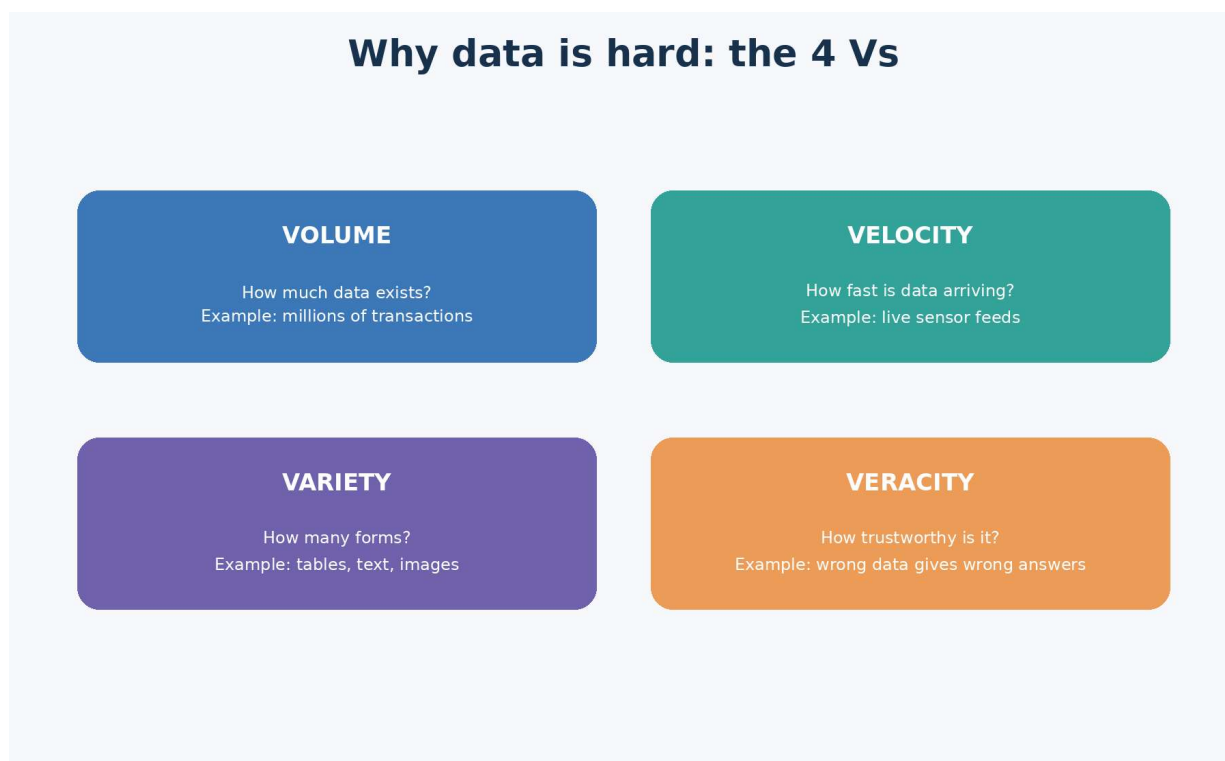


Diagram 4. The four Vs of practical data work: volume, velocity, variety, veracity.

8. The AI / ML block: four broad kinds of questions

Once data has been properly prepared, AI or machine learning can be pointed at one of several broad jobs. Classification predicts a category — which bucket does this belong in? Regression predicts a number — how much, or how many? Clustering finds natural groups in the data without ever being told what the "correct" groups are supposed to be. Generative AI creates new content — text, images, or summaries — rather than predicting a category or a number at all.

Knowing which of these four you actually need is one of the most important beginner skills in this whole course, because a lot of early mistakes come from reaching for the wrong tool. "Is this patient stable or deteriorating?" is classification. "How many days until this wound heals?" is regression. "Are there natural groups of similar patients hiding in this data that we haven't labelled yet?" is clustering. Generative AI is a different job entirely — its output isn't a category or a number, it's new content, and ideally that content stays grounded in the real data it was built from.



Diagram 5. Four broad kinds of AI questions: classification, regression, clustering, generative AI.

9. The application block, and the feedback loop that closes it

A prediction only becomes valuable once it's placed inside a real workflow that someone actually uses. In a hospital, a risk score might turn into a bedside alert for a nurse. In retail, it might become a personalised product recommendation. In manufacturing, it might trigger a maintenance ticket before a machine fails. In banking, it might block a suspicious transaction before it goes through.

The lecture then adds one more crucial idea that beginners often skip past: feedback. After a prediction gets used, you need to go back and check it against what actually happened. Did the fraud alert really catch fraud, or was it a false alarm? Did the wound genuinely deteriorate the way the model expected? Did the maintenance alert actually prevent a breakdown? Without this feedback step, there's no way to know whether the system is improving or quietly getting worse over time.

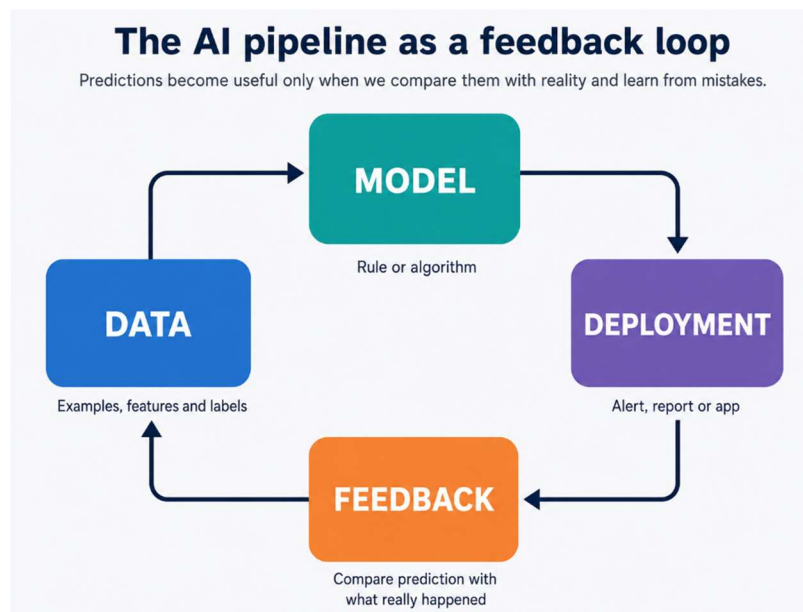


Diagram 6. The full pipeline as a feedback loop, not a one-way street.

10. Getting Python, VS Code, and your working environment ready

This lecture also covers the software setup, and while it feels like the most "technical" part, the underlying idea is genuinely simple once you see it as layers stacked on top of each other.

Python is the language itself — the actual set of words and grammar rules you're typing. Packages such as pandas and NumPy are toolboxes written by other people that you can pull into your own work, so you're not reinventing basic tools like "read this spreadsheet" from scratch every time.

VS Code is the workspace where you actually write and run your work — think of it as the desk, not the language.

A **virtual environment** is a private toolbox that belongs to just one project, so that the specific tool versions one project needs never clash with what a different project needs. Two projects can quietly disagree about which version of a tool they want, and a virtual environment keeps them from ever finding out.

The terminal is a plain text window where you type instructions directly to the computer and it types back — no buttons, no menus, just typed commands and typed replies. It looks intimidating at first mainly because it's unfamiliar, not because it's actually complicated.

PATH is the computer's internal list of "places I already know how to find programs." When you install Python and it isn't added to this list, the terminal has no idea Python exists, even though it's sitting right there on the hard drive — this is the single most common beginner setup problem, and it has nothing to do with anything you did wrong in your own code.

Jupyter Notebook is the interactive page where you actually run your work piece by piece and see the results appear immediately underneath. The **kernel** is the invisible engine running behind that page — it's the thing that actually understands Python and does the work; the notebook is just the page you're looking at while it happens.

None of this needs to feel like "real programmer" knowledge to understand conceptually. The whole idea boils down to one sentence: professional work goes more smoothly when your project has its own clean, private workspace and its own clearly controlled set of tools, instead of everything piled together in one shared mess.

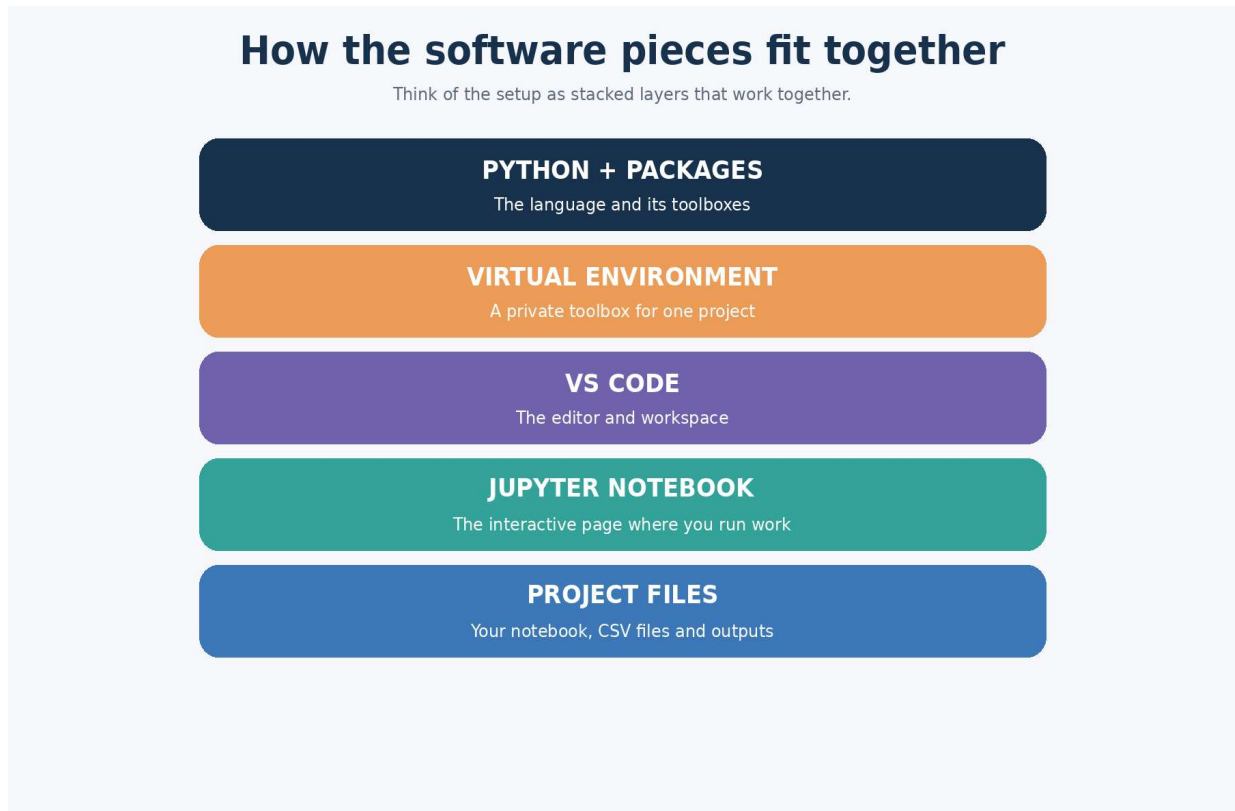


Diagram 7. How the software pieces — Python, packages, VS Code, environment, Jupyter — relate to one another.

11. Basic Python thinking

The lecture introduces six building blocks: variables, lists, dictionaries, loops, conditionals, and functions. None of these need technical definitions to make sense. A variable is a named box that holds a value. A list is an ordered collection of things, like a shopping list where order matters. A dictionary is a labelled collection, like a small record card where every piece of information has its own named slot. A conditional is a yes-or-no decision branch — if this is true, do one thing, otherwise do another. A loop repeats the same piece of work across many items, one after another, without you having to write it out by hand each time. A function is a reusable bundle of instructions you can call on whenever you need that particular job done again.

You don't need to treat these as exotic programming concepts on day one — they're just formal names for logical operations people already do mentally, all the time. "If the temperature is high, flag this patient" is already a conditional rule, running inside a nurse's head, long before it ever becomes a line of code.

Python building blocks in plain language

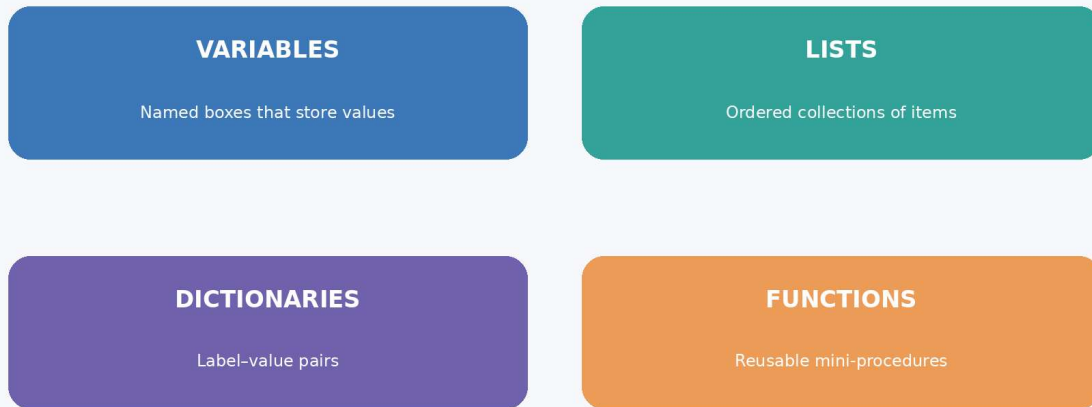


Diagram 8. Python's basic building blocks.

12. Data handling basics: rows, columns, features, and labels

With the coding basics in place, the lecture shifts into tabular data specifically. A row is one single observation — one patient visit, one sale, one sensor reading. A column is one type of information recorded about every row, such as temperature or price. Features are the input columns you use to make a prediction. The label is the answer column — the thing a model is actually trying to predict, or the thing you compare its guess against afterwards.

This sounds almost too simple to dwell on, but it's the foundation nearly everything later depends on. Mix up rows and columns, or mix up features and the label, and every machine-learning idea that follows becomes confusing for reasons that have nothing to do with the model itself.

How to read a table

Rows, columns, features and labels

patient_id	temperature	pH	status	prediction
1	36.4	7.08	stable	stable
2	36.6	7.01	stable	stable
9	37.5	7.50	deteriorating	stable

A row = one patient or observation

A column = one kind of information

Features = input columns used to make predictions

Label = the answer we compare against, such as the status

Diagram 9. Table anatomy: rows, columns, features, and labels.

13. What a CSV file actually is, and why pandas matters

A CSV file is about as simple as a text file gets: it stores a table as plain text. The first line usually holds the column names, each following line is one row, and commas separate the individual values — CSV literally stands for "comma-separated values." This course uses a small wound-care dataset specifically so you can look at every single row yourself, rather than treating the table as a mysterious black box you're told to trust.

Pandas is the table-handling toolbox for Python. In plain terms, it gives you a way to open a table, look inside it, add new columns, remove columns you don't need, save a changed version back to disk, and generally work with the data the way you'd work with a spreadsheet — just with typed instructions instead of mouse clicks.

The single most important habit this section teaches is "inspect before you trust." Before any data goes anywhere near an AI system, a responsible analyst checks the number of rows and columns, confirms the column names are what they should be, checks whether anything important is missing, and sanity-checks whether the numbers actually look plausible.

A CSV is a simple text file that stores table data

```
patient_id,temperature,pH,status  
1,36.4,7.08,stable  
2,36.6,7.01,stable  
3,36.7,7.24,stable  
9,37.5,7.50,deteriorating  
10,37.6,7.53,stable
```

Header row = column names; every later line = one row of values separated by commas.

Diagram 10. A CSV file as simple, plain table text.

14. NumPy and thinking in numbers

NumPy is the number-handling toolbox. It's built around arrays — collections of numbers specifically designed for fast, efficient numerical work. In beginner terms: NumPy is what you reach for when you want to do the same calculation to many numbers at once, instead of working through them one at a time by hand.

This section also introduces correlation — the question of whether two variables tend to move together. If temperature and pH tend to rise together, they have a positive relationship. Importantly, the lecture is careful to point out they don't move together perfectly. That imperfection matters, because real-world outcomes are almost always shaped by several factors interacting together, not just one single measurement acting alone.

Correlation asks whether two variables move together

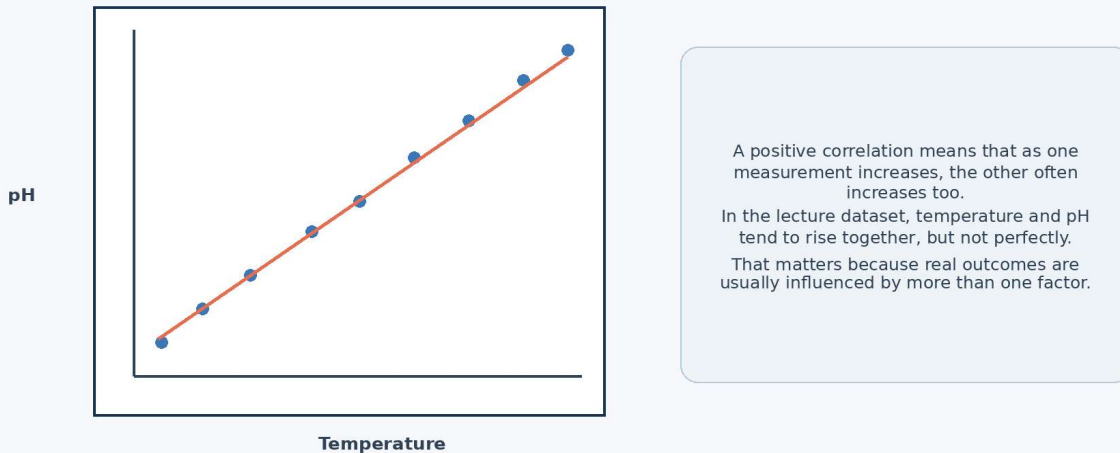


Diagram 11. A picture of positive correlation.

15. Creating new features from raw measurements

Feature creation means taking raw measurements and reshaping them into more useful inputs. A new feature might be a simple yes-or-no flag, like "is this a high temperature reading?", or it might combine several measurements into one blended score.

The core idea here is that AI systems usually don't perform their best when handed completely raw data. They tend to do better when a human has already helped shape the raw material into features that carry more obvious meaning. This shaping process is sometimes called feature engineering.

Real-life example: in banking, rather than using a raw transaction amount by itself, an analyst might build a feature like "number of transactions in the last hour," because that pattern of behaviour can reveal fraud risk far more clearly than any single transaction amount could on its own.

16. Classification: predicting a category

The lecture's first hands-on model example is classification. The classroom rule looks at a patient's temperature and pH, then predicts either "stable" or "deteriorating." The important theoretical idea here isn't the exact numbers used as thresholds — it's the general concept of a decision rule that maps input features onto an output category.

A key warning that comes with this section: accuracy on its own is not enough. A model can post a perfectly respectable overall percentage while still making mistakes that matter a great deal. A false positive means the system raises an alarm when there wasn't actually a problem. A false negative means the system stays quiet when there really was a problem. In many real settings — healthcare especially — a false negative is the far more serious of the two, even though both count equally toward a simple accuracy score.

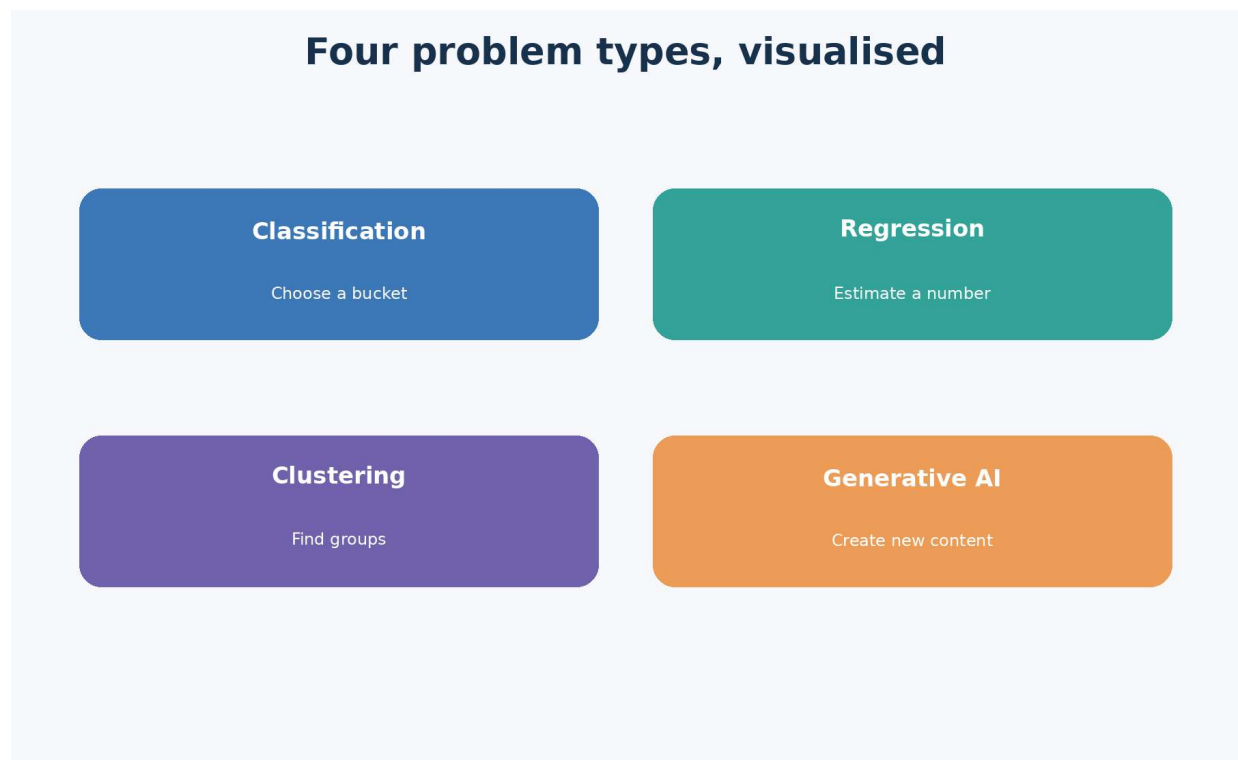


Diagram 12. A second visual summary of the four AI problem types.

17. Saving, adding, dropping, and editing a table

Once a table has been loaded and inspected, the lecture covers the everyday mechanics of actually working with it — because a real analysis rarely stops at just looking. When a file is saved again after changes, the updated table gets written back to disk, replacing (or sitting alongside) the original. This is the same idea as "Save" in any other program — nothing magical happens behind the scenes.

Adding a new column means attaching a new piece of information to every row at once — for example, calculating a risk score from existing measurements and storing it as its own column. Dropping a column means removing one that isn't needed any more, which keeps the table focused on what actually matters for the task at hand. Editing a single cell means changing one specific value directly — including deliberately marking a value as missing, which is a useful way to practise handling missing data safely before encountering it for real.

These actions matter conceptually because they make one thing very clear: a CSV file is not some special, mysterious object. It's simply a file that represents a table. An analyst opens it, changes the version sitting in the computer's memory, and saves a new version to disk whenever it's actually needed — the same basic loop, over and over.

One smaller concept worth knowing is `ignore_index`, which comes up when combining two tables together. In plain terms: when you stack one table on top of another, the row numbers (the index) can either carry over from the original tables or get reset to run cleanly from zero. `ignore_index` only changes those row number labels — it never changes the actual data sitting inside the rows. It's a bit like renumbering the pages after stapling two documents together: the page numbers change, but the content of each page doesn't.

Generative AI should be grounded in real data



Key idea: generated text is not automatically true. It must be checked by a person, especially in healthcare.

Diagram 13. What `ignore_index` changes, and what it deliberately leaves alone.

18. Regression: predicting a number

Regression is the task of predicting a numerical value rather than a category. In the lecture's example, that number is healing days. Conceptually, this is the model trying to estimate "how much" or "how many," rather than sorting something into one bucket or another.

Real-life examples include predicting house prices, forecasting energy demand, forecasting sales, or estimating a student's final exam mark. Seeing this alongside classification helps make one thing click: not every AI output looks like a yes/no answer — plenty of the most useful ones are just numbers.

19. Clustering: finding groups without being told the answer

Clustering is described as unsupervised learning — meaning, in plain terms, that nobody tells the algorithm what the "correct" groups are supposed to be ahead of time. There are no answer labels to check against at all. Instead, the algorithm looks purely at how similar or different the rows are to each other, and sorts them into groups based on that similarity alone.

The lecture uses K-means as its example clustering method. The key theoretical point to hold onto: clusters are not automatically "the truth." The algorithm finds statistical groupings based on patterns in the numbers — but a human still has to look at each group afterwards and work out what it actually means in the real world. The algorithm has no idea what to call the groups it finds.

Real-life example: a retailer might cluster customers by their spending and visiting behaviour. Afterwards, a human analyst might label the resulting groups "high-value loyal customers" or "occasional bargain shoppers" — but those names come from human interpretation after the fact, not from the algorithm itself.

20. Generative AI: turning data into a draft summary

The lecture's generative AI example takes a single row of data and turns it into a structured prompt intended for a text-generating model. The key conceptual lesson here is that simply preparing a well-structured prompt is not the same thing as actually asking a model to generate a response — the two are separate steps. More importantly, even once generative AI is used, its output has to stay grounded in real data and be checked over by a human being before anyone trusts it.

This matters even more in sensitive settings like healthcare. A generated summary should be treated the way you'd treat a first draft written by a junior colleague — useful as a starting point, but not something to accept unquestioned. The lecture uses the word non-diagnostic for exactly this reason: it's another way of saying "this text is a draft aid to support a decision, not a medical diagnosis on its own, and it still needs a qualified person to check it."

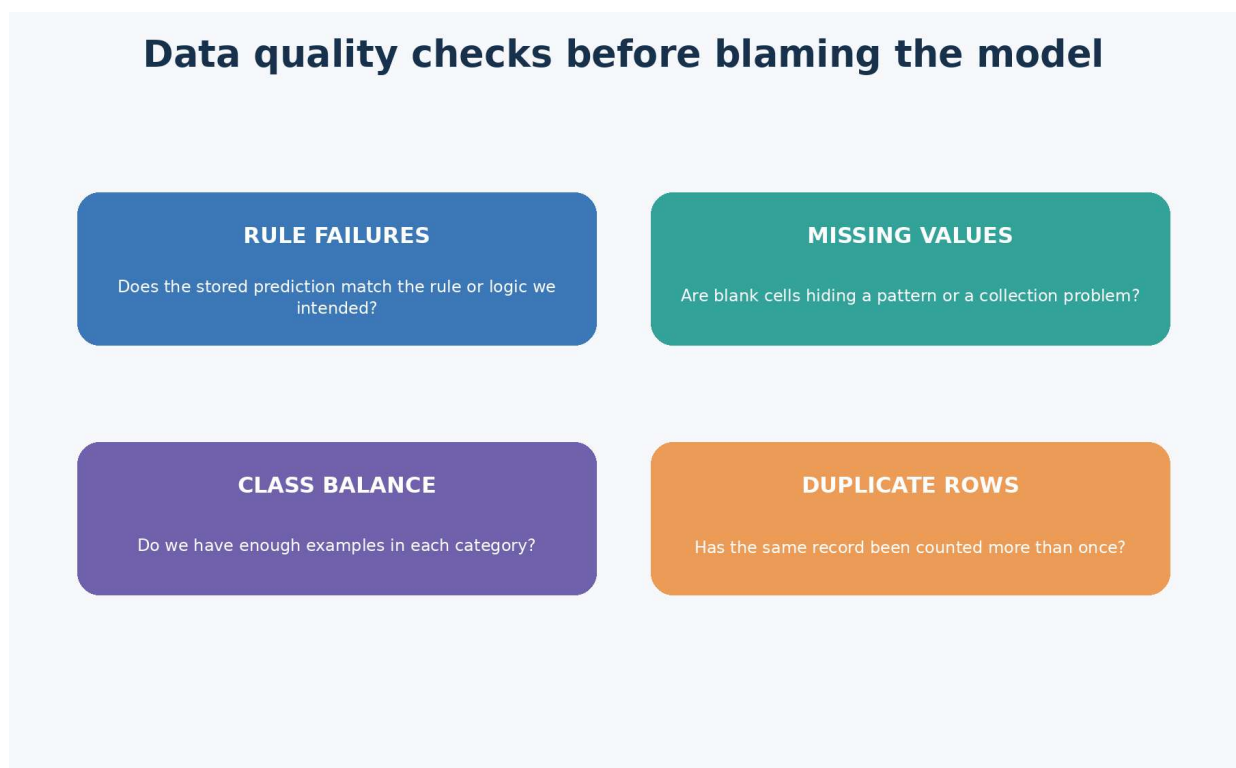


Diagram 14. Generative AI output should be grounded in real data and reviewed by a human.

21. Data-centric AI: better data beats a fancier model

This is one of the strongest messages in the entire lecture, so it's worth slowing down for. Data-centric AI means putting your effort into improving the data and the labels first, before rushing to try a more complicated model. The lecture's argument is that a great many modelling problems are actually data problems wearing a disguise — the model isn't broken, the data it was given is.

Four specific checks are highlighted as the starting point for this mindset, and it's worth walking through each one separately, since each catches a different kind of problem.

21.1 Rule failures

A rule failure means going back and checking whether a stored prediction actually matches the rule it was supposed to follow. In plain terms: pick a row where the system got it wrong, and actually look at it. Was the measurement itself unusual? Was the rule too simple to capture what was really going on? Looking closely at individual mistakes, rather than only looking at an overall accuracy number, is often where the most useful insight comes from.

21.2 Missing values

Checking missing values means asking whether blank cells in your data might be hiding a pattern, rather than being purely random gaps. Sometimes data is missing completely at random — but sometimes it's missing for a reason connected to the very thing you're trying to predict, and that connection can quietly bias your results if it goes unnoticed.

21.3 Class balance

Class balance means checking whether one category shows up far more often than another in your data. This matters because a lopsided dataset can make a system look accurate on paper while actually being close to useless in practice.

Real-life example: if 95% of loan applications in a training dataset are "approved" and only 5% are "rejected," a system that lazily predicts "approved" almost every single time will still score a 95% accuracy — despite never genuinely learning to spot the risky 5%. That's exactly why checking class balance matters before trusting an accuracy number at face value.

21.4 Duplicate rows

Checking for duplicate rows means asking whether the same record has accidentally been counted more than once. A duplicated row silently gives that particular observation extra weight in any analysis or model built on top of it, which can quietly distort results without any obvious warning sign.

Taken together, these four checks are a genuinely practical starting toolkit: before reaching for a more complex model, look at your mistakes individually, check what's missing and why, check whether your categories are balanced, and check whether anything has been counted twice.

What ignore_index changes in a duplicated table

Only the row numbers change; the duplicated patient data remains duplicated.



Diagram 15. Four essential data-quality checks: rule failures, missing values, class balance, duplicate rows.

22. Summary and the big takeaway from Lecture 1

Lecture 1 is not really about installing software or working through one small practice dataset — those are just the vehicle. Its deeper goal is to reshape how you think about this whole subject from day one. The core message is that AI is a workflow built on top of solid data engineering, not an isolated, self-sufficient model. Python and its packages are simply the tools that let you express that workflow clearly and repeatably.

By the end of this lecture, you should be able to describe the overall pipeline, name the main AI problem types, explain basic table concepts in your own words, and explain why data-quality checks come before falling in love with any particular model. That mindset will carry you further through this course — and through real work afterwards — than memorising a handful of lines of code ever will.

Appendix: how to use these notes

- When revising, try explaining each diagram out loud in your own words, without looking at the caption first.
- If a concept still feels abstract, pair it with a real-world example from healthcare, banking, retail, or manufacturing — the same four keep showing up throughout this course on purpose.
- When you get to the code later, remember the code is just the formal, typed expression of the concepts already explained here in plain language — the concept always comes first.